Issue Triage Assistant Project Document

Product, architecture, data and local operating guide

Executive summary

The Issue Triage Assistant helps auditors find relevant past issues, controls and policy, then prepare a cited recommendation for human review. The V1 application combines Streamlit, LangChain, Pinecone and LangGraph. It pauses for an auditor decision and records the review trail. It does not make final severity decisions, judge evidence sufficiency, close issues or file them into another system.

Project item	Current V1 state
Purpose	Cited issue triage and RAU scoping support for auditors.
Data boundary	Synthetic client, issue, control and policy examples only.
Vector corpus	60 past issues, 36 control records and 10 policy chunks indexed in Pinecone.
Local database	SQLite audit store at data/issue_triage.db; Postgres is available in the hosted Compose scaffold.
Human control	Auditor approves, edits or rejects; rejection requires a reason.

Operating principle The assistant drafts and cites. The auditor decides.

Product behavior and workflow

Single-issue triage

1. The auditor enters a title, description, client/audit context and optional control hint.
2. The graph recalls only approved scoped memory patterns, then retrieves past issues, controls and policy from Pinecone.
3. The model drafts severity, root cause, control mapping, duplicate warning, confidence and remediation steps.
4. A validator checks each supported claim against retrieved chunk IDs, source IDs, source types where applicable, and exact quotes. One repair attempt is allowed; weak retrieval or failed validation produces an insufficient-evidence state.
5. A durable LangGraph interrupt pauses for approve, edit or reject. Edited claims are validated again before the decision is saved.

RAU Scoping Mode

Scoping Mode retrieves by client, audit, RAU and control, applies a two-year issue lookback, and groups precedent by control, attribute and root cause. Older matching issues and control-version mismatches are surfaced. Each cluster requires an auditor decision and reason.

System architecture

Component	Responsibility in this project
Streamlit	Synthetic-data intake, cited result display, auditor decision forms and run history.
LangChain	OpenAI chat/embedding clients, structured output and Pinecone vector-store integration.
LangGraph	Single-issue state machine, citation repair route, durable approval interrupt and scoping review loop.
Pinecone	Semantic retrieval in past_issues, controls and policies namespaces; tenant/audit metadata filters precede ranking.
SQLAlchemy + SQLite/Postgres	Issue/run snapshots, retrieved evidence, append-only events, decisions and scoping packs.
Mem0 (optional)	Auditor/client-scoped recall and write-back of compact patterns only after approval or edit.

Triage flow

Intake	Retrieve	Draft	Validate	Human review	Audit record
Issue + context	Issues, controls, policy	Structured claims	Exact source checks	Approve / edit / reject	Decision + events

Data design and audit trail

Pinecone source records

Pinecone holds searchable source text plus filterable metadata. SQL remains the application record for submitted runs and human decisions. The ingestion command creates deterministic chunk IDs so repeated ingestion updates the same synthetic source records.

Namespace	Semantic text	Useful metadata
past_issues	Title, condition, scope, severity, root cause, control, remediation, outcome and status.	client_id, audit_id, issue_id, control_id, dates, RAU, attribute, status.
controls	Control objective, owner/action, frequency, evidence and exception handling.	client_id, audit_id, control_id, control_version, RAU, attribute.
policies	Policy sections split into overlapping text chunks.	client_id, audit_id, policy ID, section, effective date and chunk ID.

Corpus note: the 60 issue rows use six recurring synthetic scenario patterns. They exercise ingestion and retrieval, but they are not a broad evaluation set.

SQL records

Record group	What it preserves
issues and triage_runs	Submitted issue snapshot, client/audit IDs, model and prompt version, retrieved chunks, suggestion and run status.
approvals	Decision, approver, reason, time and edited suggestion where applicable.
audit_events	Graph events, retrieval/model timing and token-use metadata when available.
scoping_runs and scoping_decisions	Saved scoping pack and per-cluster decision, reason and approver.

Memory policy

When Mem0 is configured, the current adapter stores only a compact accepted pattern, such as an approved severity category, root-cause category and control ID. The memory key is scoped to auditor and client; metadata records the source issue ID and approver. Raw descriptions, retrieved evidence, unapproved drafts and secrets are not written to memory.

Citation and safety boundaries

- Exact quote validation confirms that a quote exists in a retrieved chunk; it does not prove the quote semantically supports the conclusion.
- A retrieval similarity threshold is a retrieval signal, not an evidence-sufficiency judgement.
- All displayed recommendations remain suggestions. There is no auto-approval, auto-close, external issue creation or auto-filing.
- The current demo has no production identity-to-tenant authorization policy. Do not use real client data until that gate is implemented and reviewed.

Local setup and operation

Run with the current OpenAI and Pinecone settings

1. Use Python 3.11 or newer and activate the project virtual environment.
2. Set OPENAI_API_KEY and PINECONE_API_KEY in the local .env file. Keep secrets out of source control.
3. The current vector contract is text-embedding-3-small with 1,536 dimensions and cosine similarity. If the model or dimension changes, create a compatible index and re-embed every source.

Index or refresh the synthetic corpus (this upserts records and does not delete the namespaces):

```
.venv/bin/python -m src.issue_triage.ingestion.cli_ingest
```

Start the UI in another terminal:

```
APP_ENV=development AUTH_MODE=demo .venv/bin/streamlit run app.py
```

Open http://127.0.0.1:8501. Use the synthetic client/audit choices. The --rebuild flag deletes the three configured Pinecone namespaces before re-indexing; reserve it for a dedicated concept index.

Configuration reference

Setting	Purpose
DATABASE_URL	Local SQL persistence; default SQLite file is data/issue_triage.db.
CHECKPOINT_BACKEND / CHECKPOINT_DATABASE_URL	Durable LangGraph state; SQLite locally, Postgres for the Compose scaffold.
PINECONE_INDEX	Target vector index; defaults to issue-triage-v1.
CHAT_MODEL / EMBEDDING_MODEL	Model IDs for structured triage and vector embedding.
RETRIEVAL_THRESHOLD / PROMPT_VERSION	Weak-match screening and run trace/version metadata.
MEM0_API_KEY	Optional approved-pattern memory; absent means memory is disabled.

Before real-client deployment

- Implement and verify identity-to-client/audit authorization on every retrieval and record access path.
- Commit a fully resolved dependency lock and validate supported runtime/provider model combinations.
- Calibrate retrieval thresholds and severity behavior against auditor-reviewed examples; add repeatable evaluation and monitoring.
- Complete security, privacy, retention, backup, recovery, vendor and operational reviews for the deployment environment.

Project files Product spec: docs/product_spec.md | Prompt: docs/system_prompt.md | Agent guidance: AGENTS.md | App: app.py